Inteligencia Artificial. Tarea 3

Tomas Santana

Octubre, 2026

La tarea se realiza en parejas.

1. Kernel cuadrático (3 puntos). Considere el kernel

$$K(\boldsymbol{x}, \boldsymbol{z}) = (\boldsymbol{x}^T \boldsymbol{z} + c)^2 = \phi(\boldsymbol{x})^T \phi(\boldsymbol{z})$$

con $c = 9$ y $\boldsymbol{x}, \boldsymbol{z} \in \mathbb{R}^2$.

- Usando la transformación ϕ vista en clase, determinar $\phi([2, -3]^T)$.
- Con $\boldsymbol{z} = [1, 4]^T$, calcular $\phi(\boldsymbol{x})^T \phi(\boldsymbol{z})$ y $K(\boldsymbol{x}, \boldsymbol{z})$ para $\boldsymbol{x} = [2, -3]^T$, y verificar que coinciden. Pueden hacerlo a mano o con `numpy`.
- Para d features, ¿cuántas componentes tiene $\phi(\boldsymbol{x})$? ¿Del orden de cuántas operaciones cuesta calcular $\phi(\boldsymbol{x})^T \phi(\boldsymbol{z})$ directamente, y cuántas cuesta calcular $K(\boldsymbol{x}, \boldsymbol{z})$?

2. SVC con kernel gaussiano (7 puntos). Descargar un dataset de datos reales con entre 1000 y 10 000 ejemplos, entre 5 y 10 features cuantitativas (no variables indicadoras) y una etiqueta categórica (puede tener más de 2 clases). Si el dataset tiene más de 10 000 ejemplos, tomar una muestra aleatoria de 10 000. El dataset debe ser distinto a los usados en las tareas anteriores y en los laboratorios. Deben poder explicar qué mide cada feature.
   - Realizar un análisis exploratorio breve que incluya la proporción de ejemplos de cada clase, y limpiar los datos según sea necesario.
   - Separar un conjunto de prueba.
   - Escoger 2 de sus features y, con ellas estandarizadas, entrenar un `SVC(kernel="rbf")` para 3 valores de `gamma` con `C` fijo, y para 3 valores de `C` con `gamma` fijo. Graficar la frontera de decisión de cada modelo y marcar los vectores de soporte. Pueden adaptar la función `graficar` del Laboratorio 03 o usar `DecisionBoundaryDisplay` de `sklearn` (si tienen más de 2 clases, basta con graficar las regiones de cada clase). Reportar el número de vectores de soporte y la exactitud de entrenamiento y validación de cada modelo, y explicar cómo cambia la frontera con `gamma` y con `C`.
   - Construir un `Pipeline` con `StandardScaler` y `SVC(kernel="rbf")`, y escoger `C` y `gamma` con `GridSearchCV` sobre una grilla de valores en escala logarítmica.
   - Graficar un mapa de calor del score de validación cruzada para cada combinación de `C` y `gamma`.
   - Reportar el score del mejor modelo sobre el conjunto de prueba una sola vez, al final.

3. Árbol de decisión y random forest (6 puntos). Sobre los mismos datos y la misma separación de prueba:
   - Escoger la profundidad máxima (`max_depth`) de un `DecisionTreeClassifier` con `GridSearchCV`.
   - Dibujar un árbol de profundidad 3 con los nombres de las features y de las clases, y explicar el proceso de decisión que describe.
   - Escoger los hiperparámetros de un `RandomForestClassifier` con `GridSearchCV` (por ejemplo, `n_estimators`, `max_depth` y `max_features`).

- Reportar el score de prueba del árbol y del random forest.

4. Evaluación y comparación (4 puntos).
   - Mostrar la matriz de confusión sobre el conjunto de prueba de los tres modelos (SVC, árbol y random forest), y reportar la precisión y la sensibilidad (recall) de cada clase.
   - Por defecto, `GridSearchCV` escoge los hiperparámetros usando el método `score` del modelo, que en clasificación es la exactitud. Explicar por qué esto puede no ser óptimo y si lo es o no para su dataset.
   - Comparar SVC y random forest, y explicar las ventajas y desventajas de cada uno en su problema.
   - Si usaron `class_weight`, explicar qué hace y cómo cambió sus resultados. Mostrar la matriz de confusión y las métricas de precisión y sensibilidad con y sin `class_weight`.

Entrega y evaluación

Cada pareja debe entregar un notebook (`.ipynb`) con la solución de todos los puntos y el dataset utilizado. La entrega es **en clase**. El notebook debe ejecutarse de principio a fin sin errores. La nota definitiva se calculará como:

$$\text{Nota} = \text{Implementación} \times \text{Explicabilidad},$$

donde la implementación es la suma de los puntos anteriores (de 0 a 20) y la explicabilidad es un factor entre 0 y 1 que mide la capacidad de presentar y justificar la solución propia. Para medirla, cada pareja responderá 3 preguntas sobre su implementación, con su código abierto y ejecutable. Cualquiera de los dos integrantes puede ser llamado a responder. El factor se asigna según la siguiente tabla:

Factor	Criterio
1.0	Responde correctamente las 3 preguntas y ubica en su código lo que se le pregunta sin dudar.
0.8	Responde 2 preguntas de forma completa y 1 de forma parcial.
0.6	Responde correctamente 2 preguntas.
0.3	Responde correctamente 1 pregunta.
0	No responde correctamente ninguna pregunta, o no logra ubicar en su propio código lo que se le pregunta.

Una respuesta es **correcta** si contiene todos los puntos esperados y **parcial** si contiene la idea principal pero le falta alguno de ellos.

Temas para preparar la evaluación

Las preguntas se escogerán de un banco que cubre los siguientes temas. Para cada uno, deben poder explicarlo con sus propias palabras y señalar dónde aparece en su código.

1. Construir la transformación ϕ del kernel $(\boldsymbol{x}^T\boldsymbol{z} + c)^2$ para cualquier vector, no solo el de la tarea (Notas 05, kernels polinomiales).
2. El número de componentes de ϕ y el costo de calcular $\phi(\boldsymbol{x})^T\phi(\boldsymbol{z})$ frente al de calcular $K(\boldsymbol{x}, \boldsymbol{z})$, y por qué eso hace útiles a los kernels.
3. La relación entre `gamma` y σ en el kernel gaussiano, cómo cambia la frontera al variar `gamma` y cómo se ve el overfitting en sus gráficas de frontera y en su mapa de calor.
4. Qué controla `C` en la SVM de margen suave, su relación con las holguras ξ_i, y cómo cambian el margen y los vectores de soporte en sus gráficas de frontera.
5. Por qué se estandarizan las features para el kernel gaussiano, y por qué el `StandardScaler` debe ir dentro del `Pipeline` y no aplicarse antes de `GridSearchCV`.
6. Cómo funciona `GridSearchCV`: cuántos modelos entrena su búsqueda, qué significa `best_score_` y por qué es distinto del score de prueba.

7. Cómo recorrer su árbol con un ejemplo concreto: qué pregunta se hace en cada nodo, en qué hoja termina, y qué devuelven `predict` y `predict_proba` en esa hoja.
8. Por qué un árbol muy profundo sobreajusta y cómo lo muestra su búsqueda de `max_depth`.
9. Bagging y random forests: qué es una muestra bootstrap, por qué promediar árboles reduce la varianza y por qué se usa `max_features` menor que el número de features.
10. Leer su matriz de confusión: calcular la precisión y la sensibilidad de una clase, decidir qué error es más costoso en su problema y qué métrica (`scoring`) usaría en lugar de la exactitud.
11. Ventajas y desventajas de SVC y random forest en su problema (escalado, interpretabilidad, costo de entrenamiento, hiperparámetros) y, si usaron `class_weight`, qué hace y cómo cambió sus resultados.